APDB301 Revision Exercise

Section A: Multiple Choice Questions (MCQs)

Answer the following MCQs. Each question has one correct answer.

1. **Which Azure feature helps manage and allocate resources efficiently based on varying demands in a cloud database setup?**

    a) Azure Traffic Manager

    b) Azure SQL Managed Instance

    c) Azure Virtual Machine Scale Sets

    d) Azure SQL Database Elastic Pools

2. **What is the role of HTTP methods (verbs) when interacting with web services?**

    a) To define the headers used in a request

    b) To determine the operation that should be performed on a resource

    c) To set the security protocol for the request

    d) To specify the structure of the URL

3. **Which HTTP status code signifies that the client needs to provide authentication to proceed with the request?**

    a) HTTP 201 Created

    b) HTTP 403 Forbidden

    c) HTTP 401 Unauthorized

    d) HTTP 503 Service Unavailable

4. **Which HTTP method should be used to modify the details of an existing resource in a web service?**

    a) PATCH

    b) PUT

    c) POST

    d) GET

5. **What does the acronym REST refer to in the context of web services?**

    a) Resource State Transfer

    b) Remote State Transition

    c) Representational State Transfer

    d) Reliable Service Transfer

6. **Which key characteristic defines RESTful architecture for web services?**

    a) Stateless interactions

    b) Layered system

    c) Cacheable responses

    d) Uniform interface design

7. **Which keyword pair is used in C# to handle asynchronous methods?**

    a) Async and await

    b) Await and begin

    c) Async and proceed

    d) Await and task

8. **In a RESTful Web API, which HTTP method is typically partially used to update a resource?**

    a) POST

    b) GET

    c) PATCH

    d) DELETE

9. **What is the advantage of using Azure SQL Database Elastic Pools?**

    a) Enables automatic backup of all databases

    b) Allows databases to share resources dynamically based on usage

    c) Improves data encryption for security

    d) Facilitates cross-region replication for disaster recovery

10. **Which feature of Azure Service Bus enables multiple subscribers to receive a copy of the same message?**

    a) Broadcast Queue

    b) Shared Subscription

    c) Topic and Subscription

    d) Multi-Channel Queue

11. **What is the purpose of using the await keyword in C#?**

    a) To pause the execution until a specific event occurs

    b) To handle exceptions more effectively in code

    c) To perform operations without blocking the main thread

    d) To specify that a function will run continuously

12. **Which of the following best describes RESTful architecture's "Layered System" constraint?**

    a) Clients must be aware of all underlying layers and infrastructure.

    b) Each layer in the architecture can only communicate with its immediate neighbour.

    c) Layers can be bypassed if a faster path is available.

    d) The architecture supports stateful sessions across multiple layers.

13. **What does the [RequireHttps] attribute do in a Web API?**

    a) Logs all request and response data for analysis

    b) Allows clients to switch between HTTP and HTTPS freely

    c) Forces communication to occur over a secure HTTPS connection

    d) Encrypts the data payloads being sent and received

14. **Which Azure service best suits managing large-scale event-based messaging between different applications?**

    a) Azure Event Grid

    b) Azure Notification Hubs

    c) Azure IoT Hub

    d) Azure Service Bus Topics

15. **When designing a Web API, which of the following is considered a best practice for versioning?**

    a) Append the version number to the domain name

    b) Use custom HTTP headers to specify the API version

    c) Include the version as a query parameter

    d) Embed the version within the URL path (e.g., /api/v2/resource)

16. **Which of the following best describes the "Client-Server" constraint in RESTful architecture?**

    a) The server stores the client state across multiple interactions.

    b) The client and server are interdependent, requiring simultaneous updates.

    c) The client and server are separate entities capable of independent development.

    d) The server can modify client requests before processing them.

17. **What does the "Stateless" constraint mean in a RESTful system?**

    a) Each request must be processed independently, without relying on previous requests.

    b) The server tracks the client's session to improve response accuracy.

    c) The client stores session information locally for reference.

    d) The server maintains a log of client interactions for security.

Section B: Short Answer Questions

1 **Difference Between GET and POST Methods:** Explain the primary differences between the GET and POST HTTP methods and provide a scenario where each would be appropriate.

Get VS Post	GET	POST
Purpose	Retrieve Data	Submit/ Create Data
Data location	Append to URL as a string	Sent in request body.
Visibility	Visible in URL, browser history, logs	Not visible in URL
Caching	Can be cached	Not typically cached
Idempotency	Idempotent (repeating has no extra effect)	Not idempotent
Data Size Limit	Limited by URL length	Effectively unlimited

- Get Scenario: Fetching a product's details for a product page – (GET/api/45).
- Post Scenario: Submitting a new user registration form with sensitive/large data – (POST/api/users).

2. **Securing Web APIs:** Identify three strategies you could implement to secure a Web API that handles sensitive data.

1. **Authentication and Authorization** – use OAUTH2/JWT tokens or [AUTHORIZE] attributes to ensure only verified users/clients access endpoints.

2. **HTTPS/TLS enforcement**- use [RequireHttps] to encrypt data in transit and prevent interception.

3. **Input validation and rate limiting –** validate/sanitize inputs to prevent injection attacks, and throttle requests to prevent abuse (eg. DDoS, brute force).

3. **Benefits of Using Asynchronous Methods:** Explain why using asynchronous methods in a web application can improve performance. Include an example scenario.

   Asynchronous methods free up the calling thread while waiting for I/O-bound operations (network calls, disk reads, database queries) to complete, rather than blocking it. This improves **scalability and responsiveness**, since the thread can serve other requests in the meantime instead of sitting idle.

   Example: A web server handling file uploads to cloud storage — using async/await with HttpClient lets the server accept and process other incoming requests while it waits for the upload to finish, instead of freezing the thread until the upload completes.

4. **Error Codes in Web APIs:** Describe the use cases for the following HTTP status codes:

- a) 500 Internal Server Error -An unexpected condition on the server prevented it from fulfilling a valid request (e.g., unhandled exception, bug in server code).
- b) 204 No Content - the request succeeded, but there is no content to return (e.g., a successful DELETE operation).
- c) 302 Found - The requested resource has been temporarily moved to a different URL; the client should follow the redirect but continue using the original URL for future requests.

5. Code Completion:

Examine the following incomplete class for a BasicAuthenticationHandler. Fill in the missing parts of the code:

```
public class BasicAuthenticationHandler : _____
{
   private const string HardCodedUsername = "admin";
private const string HardCodedPassword = "password";
private const string Realm = "MyWebSite";

   protected override Task<AuthenticateResult> HandleAuthenticateAsync()
   {       if
(!_____())
      {
         return Task.FromResult(AuthenticateResult.Fail("Authorization header missing."));
      }
   }

}
```

```
public class BasicAuthenticationHandler :
AuthenticationHandler<AuthenticationSchemeOptions>

{

   private const string HardCodedUsername = "admin";

   private const string HardCodedPassword = "password";
```

```
    private const string Realm = "MyWebSite";



    protected override Task<AuthenticateResult> HandleAuthenticateAsync()

    {

        if (!Request.Headers.ContainsKey("Authorization"))

        {

            return Task.FromResult(AuthenticateResult.Fail("Authorization header missing."));

        }



        // ... decode and validate credentials here

        return Task.FromResult(AuthenticateResult.Success(/* ticket */));

    }

}
```

6. Azure Service Bus:

a) What happens when you set AutoCompleteMessages to true in Azure Service Bus?

The processor automatically marks a message as complete (removes it from the queue) once the handler finishes successfully, without requiring an explicit CompleteMessageAsync() call. If the handler throws an exception, the message is abandoned instead.

b) What happens when you set the ReceiveMode to ReceiveAndDelete?

The message is deleted from the queue **as soon as it is received**, before processing finishes. This is faster but riskier — if processing fails, the message is already gone and cannot be retried (no lock/complete/abandon cycle).

7. **HTTP Status Codes:** Explain the differences between the following HTTP status codes:

- a) 200 OK - Request succeeded; the server returns the requested resource/result.
- b) 404 Not Found - The requested resource does not exist at the given URL.
- c) 401 Unauthorized - The client has not provided valid authentication credentials; it must authenticate first.
- d) 403 Forbidden - The client is authenticated but does not have permission to access the resource.

6 **Basic Authentication:** How can you ensure that a Web API only allows access to authenticated users using the [Authorize] attribute in C#? Provide an example.

Decorate a controller or action with the [Authorize] attribute so unauthenticated requests are automatically rejected with a 401 response:

```
[Authorize]

[ApiController]

[Route("api/[controller]")]

public class OrdersController : ControllerBase

{

  [HttpGet]

  public IActionResult GetOrders()

  {

    return Ok(orders); // only reached if user is authenticated

  }

}
```

The framework checks the request against the configured authentication scheme (e.g., the BasicAuthenticationHandler above) before allowing the request to reach the action method.

6. **Cross-Database Queries in SQL Azure:** Discuss the limitations of cross-database queries in SQL Azure and how these can be managed.

Azure SQL Database does not natively support cross-database JOIN queries the way on-premises SQL Server does, because each database can potentially live on a different physical server/node. Limitations include:

- No direct three-part naming (Database.Schema.Table) across databases in the same query.
- No native distributed transactions across databases.

Workarounds:

- **Elastic Query** — query across databases using external tables.
- **SQL Data Sync** — replicate needed data into a single database.
- Move to **Azure SQL Managed Instance**, which supports cross-database queries within the same instance.
- Use an ETL/integration layer (e.g., Azure Data Factory) to consolidate data.

7. **Security in Web APIs:** List and briefly explain three methods for securing Web APIs.

1. **Authentication/Authorization** (OAuth2, JWT, API keys) — verify identity and permissions.
2. **Transport-level encryption (HTTPS/TLS)** — protect data in transit from interception.
3. **Input validation & throttling/rate limiting** — prevent injection attacks and abuse/DoS.

Section C: Programming

1. Async Programming:

Write an asynchronous method that performs the following operations:

1. **Read URLs from a Text File**: Read a list of URLs from an input text file (urls.txt). Each URL should be on a separate line.
2. **Fetch Content from Each URL**: Use HttpClient to fetch the content from each URL asynchronously.

3. **Save Responses to Separate Files**: Save each response to separate text files named response1.txt, response2.txt, etc., based on the read order.
4. **Log the Operation Status**: Create a log file (operation_log.txt) to record the status of each operation. For example, if a URL fetch was successful, log the message "Successfully fetched content from [URL]". If there was an error, log "Failed to fetch content from [URL] - [Error Message]".
5. **Handle Errors Gracefully**: Ensure that the method handles scenarios where a URL might be unreachable or invalid by catching exceptions and continuing to process the remaining URLs.

Specifications:

- Use asynchronous methods to perform all file I/O and HTTP operations.
- Ensure non-blocking file writing and reading.
- Log each operation result in operation_log.txt.

```
public async Task FetchAndSaveUrlsAsync(string inputFilePath)

{

   // Your implementation here

}
```

Azure Service Bus and OOP:

Explain what the following line of code does in an Azure Service Bus example:

```
_processor.ProcessMessageAsync += MessageHandler;
```

Service Bus

Examine the following code that handles processing messages from an Azure Service Bus queue and answer the questions that follow:

```
public class QueueProcessor

{
```

```
    private static ServiceBusClient _client;
private static ServiceBusProcessor _processor;

    public static void Initialize(string connectionString, string queueName)
    {
        if (_client == null)
        {
            _client = new ServiceBusClient(connectionString);
            _processor = _client.CreateProcessor(queueName, new ServiceBusProcessorOptions
            {
                AutoCompleteMessages = false,
                MaxConcurrentCalls = 5,
                ReceiveMode = ServiceBusReceiveMode.PeekLock
            });

            _processor.ProcessMessageAsync += MessageHandler;
            _processor.ProcessErrorAsync += ErrorHandler;
        }
    }

    private static async Task MessageHandler(ProcessMessageEventArgs args)
    {
        string body = args.Message.Body.ToString();
        Console.WriteLine($"Received message: {body}");
```

```
    // Complete the message after processing
await args.CompleteMessageAsync(args.Message);
  }

  private static Task ErrorHandler(ProcessErrorEventArgs args)
  {
    Console.WriteLine($"Error occurred: {args.Exception.Message}");

    return Task.CompletedTask;
  }
}
```

Questions:

1) What is the purpose of setting AutoCompleteMessages to false in the ServiceBusProcessorOptions?

AutoCompleteMessages = false: The message is **not** automatically removed from the queue when the handler finishes. The developer must explicitly call args.CompleteMessageAsync(args.Message) (as done in the code) after successful processing. This gives fine-grained control — if processing fails, the message can instead be abandoned/dead-lettered and retried, rather than being lost.

2) Explain what happens when the MaxConcurrentCalls is set to 5.

MaxConcurrentCalls = 5: Up to **5 messages can be processed concurrently** by the processor at any given time (i.e., 5 parallel invocations of MessageHandler). This increases throughput but also means the application must handle concurrent processing safely (thread-safety of shared resources).

Advance C#:

1. **Async Method Implementation:** Write an asynchronous method to download a file from a given URL and save it to a local directory. Use proper error handling to manage cases where the download might fail.

```
public async Task DownloadFileAsync(string url, string savePath)
{
```

```
    using var httpClient = new HttpClient();
    try
    {
        byte[] fileBytes = await httpClient.GetByteArrayAsync(url);
        await File.WriteAllBytesAsync(savePath, fileBytes);
        Console.WriteLine($"Downloaded successfully to {savePath}");
    }
    catch (HttpRequestException ex)
    {
        Console.WriteLine($"Download failed: {ex.Message}");
    }
    catch (Exception ex)
    {
        Console.WriteLine($"An unexpected error occurred: {ex.Message}");
    }
}
```

2. **C# Class Design:** Design an introductory C# class named Product, which includes properties for ID, Name, Price, and Stock. Write a method to update the stock asynchronously when new items are added to the inventory.

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
    public int Stock { get; set; }

    public async Task UpdateStockAsync(int quantityToAdd)
```

```
    {
        // simulate async work, e.g., a database update
        await Task.Delay(100);
        Stock += quantityToAdd;
        Console.WriteLine($"Stock updated for {Name}: new stock = {Stock}");
    }
}
```

3. **SQL Azure:** Discuss how you would set up an Azure SQL Database with automatic scaling to handle peak traffic times. What configuration settings would you enable, and why?

Use the **vCore-based purchasing model** with:

- **Serverless compute tier** — automatically scales compute up/down (and pauses during inactivity) based on workload, ideal for unpredictable/peak traffic.
- **Autoscale storage** — grows storage automatically as data increases.
- **Elastic Pools** — if managing many databases with variable, non-simultaneous peak loads, pool them to share resources rather than over-provisioning each individually.

These settings are chosen because they let the database automatically adjust vCores/DTUs to match demand, avoiding both over-provisioning cost during quiet periods and performance bottlenecks during peaks.

4. **RESTful Principles:** Provide an example of a poorly designed Web API endpoint and explain why it violates RESTful design principles. Then, suggest a better approach.

Bad design:
GET /getUserById?id=5
POST /deleteUser?id=5
Why it violates REST:

- Uses a verb (getUserById, deleteUser) in the URL instead of letting the HTTP method convey the action (violates **uniform interface**).
- Uses POST for a delete operation instead of the semantically correct DELETE method.
- Resource isn't represented as a noun/collection.

Better approach:
GET /api/users/5 → retrieve user 5
DELETE /api/users/5 → delete user 5

This treats users as a resource collection, uses nouns in the URL, and lets the HTTP method express the action — consistent with REST's uniform interface principle.